

BackEnd:

Para el backend se va a utilizar MVC y las capas serian:

Controller:

Este va a recibir las peticiones y valida datos básicos o campos vacíos y también retorna las respuestas de lo que se solicita.

Services:

En services se maneja la lógica de negocio y es el intermediario entre los request de controller y el manejo de datos con Repository a la BD.

Repository:

Maneja el acceso a base de datos e interactúa con la BD, cuando se solicita algo en controller pide a service y este le va a decir a Repository que le brinde la información.

Model:

Representa las entidades del sistema, los objetos.

Flujo de la petición:

Cliente (App) -> Controller -> Service -> Repository -> Base de Datos -> Repository -> Service -> Controller -> Response (JSON)

Explicacion del flujo:

La app envia la solicitud y el controller recibe el request, después el controller enviar al service para guarda la información(depends del tipo de request) services valida que cumpla con las reglas de negocio y si las cumple pasa al repository para ser almacenado en base de datos, y este devuelve un 200 o 201 en caso de éxito, también si no cumple las reglas no llega a la capa de repository y devuelve mensaje con el error encontrado.

Justificar las decisiones tomadas:

La arquitectura backend se diseñó utilizando el patrón MVC por capas (Controller, Service, Repository y Model) con el objetivo de lograr un sistema organizado, mantenible y escalable. Esta separación permite que cada componente tenga una responsabilidad específica: el Controller se encarga de manejar las solicitudes HTTP y respuestas, el Service contiene la lógica de negocio y validaciones del sistema, el Repository gestiona el acceso a la base de datos, y el Model define la estructura de los datos. Esta decisión evita el acoplamiento entre capas, facilita la reutilización de código y mejora la claridad del sistema. Además, permite realizar cambios o agregar nuevas funcionalidades, como autenticación o pagos, sin afectar todo el sistema. En conjunto, esta arquitectura asegura una mejor organización del código, facilita el mantenimiento y prepara el sistema para futuras integraciones o escalabilidad.